

Project Logisch Programmeren

Analyse Tool voor Vereenvoudigde WebAssembly in Prolog

Nathan Vermeiren

3 mei 2026

1 Inleiding

Dit project heeft als doel het implementeren van een interpreter en analyse tool voor een vereenvoudigde versie van WebAssembly in Prolog. De tool ondersteunt drie modi:

- **run**: het uitvoeren van een programma
- **analyse**: het vinden van inputs die leiden tot een trap
- **paths**: het vinden van inputs voor elk uitvoeringspad

Dit project leent zich uitstekend tot logisch programmeren omdat symbolische uitvoering, backtracking en padverkenning hier natuurlijke toepassingen van zijn.

In dit verslag wordt eerst de algemene architectuur besproken, daarna de werking van de interpreter, gevolgd door de analyse engine en tenslotte de teststrategie en conclusies.

2 Algemene Architectuur

Het project bestaat uit vier grote componenten:

- Parser voor het pwat formaat en dit omzetten naar een interne prolog structuur.
- Interpreter voor concrete uitvoering.
- Analyse engine voor het vinden van goede en slechte inputs.

Het is handig om nu al te vermelden dat de interpreter en de analyse logica heel nauw samen liggen, en heel veel dezelfde predicaten gebruiken. Dit is logisch omdat het analyseren ook code moet gaan 'uitvoeren'.

3 Parser

De parser leest een pwat bestand en zet dit om naar een interne Prolog representatie die eenvoudig kan geïnterpreteerd worden. Hoe werkt dit concreet in mijn project:

Alle parsing bevindt zich in `src/parser.pl`. Eerst lezen we het volledige bestand in als string. Deze string wordt vervolgens omgezet naar een lijst van karaktercodes. Op deze lijst van codes passen we een **DCG** (Definite Clause Grammar) toe.. De DCG doorloopt dus letterlijk elk karakter van het bestand en probeert dit te herkennen volgens de grammatica die gedefinieerd is in de parser. We doen dit op de volgende manier: De entry point van de parser is de DCG-regel `module/1`. Deze verwacht dat het bestand begint met een `(module ...)` constructie. Van hieruit wordt het bestand verder opgesplitst in drie grote delen:

- de startfunctie
- eventuele data segmenten
- de lijst van functies

Vooraleer effectieve instructies of structuren gelezen worden, wordt telkens eerst de DCG-regel `ws/0` aangeroepen. Deze regel verwijdert alle witruimte en commentaar. Commentaar wordt herkend aan de prefix `';;'` en wordt volledig genegeerd tot aan het einde van de lijn. Hierdoor kan de rest van de grammatica ervan uitgaan dat irrelevante tekens reeds verwijderd zijn.

¹

Elke syntactische constructie uit het `pwat` formaat krijgt vervolgens een eigen DCG-regel:

- `start/1` leest de startfunctie
- `data_segment/1` leest data segmenten
- `function/1` leest functies met hun eigenschappen
- `instruction/1` leest individuele instructies

Een functie wordt geparsed naar een Prolog term van de vorm:

```
func(Args, Locals, Results, Instrs)
```

waarbij `Instrs` een lijst is van instructietermen.

Elke WebAssembly instructie wordt rechtstreeks omgezet naar een Prolog term met dezelfde semantiek. Bijvoorbeeld:

- `i32.const 3` wordt `i32.const(3)`
- `i32.add` wordt `i32.add`
- `local.get 0` wordt `local.get(0)`

Control flow instructies zoals `block`, `loop` en `if` worden recursief geparsed. De instructies die zich binnen zo'n blok bevinden worden opnieuw verwerkt door dezelfde `instruction/1` regel. Hierdoor ontstaat een **geneste boomstructuur** van instructies in plaats van een platte lijst. Dit is belangrijk voor de interpreter, aangezien de blokstructuur expliciet beschikbaar blijft in de representatie.

Bijvoorbeeld, een `block` wordt omgezet naar:

```
block(Instrs)
```

waarbij `Instrs` opnieuw een lijst van instructies bevat.

Voor oproepen van functies wordt onderscheid gemaakt tussen:

- interne functies, aangeduid met een index
- primitieve functies, aangeduid met een naam zoals `read_int` of `print_int`

Deze worden beide omgezet naar de term `call(X)` waarbij `X` ofwel een getal ofwel een atoom is. Ten slotte worden gehele getallen herkend via de regels `integer/1` en `digits/1`. String literals uit data segmenten worden letterlijk opgeslagen als strings zonder verdere escape-verwerking. Dit is voldoende aangezien de interpreter later deze strings byte per byte kan verwerken.

Het resultaat van de parser is uiteindelijk een Prolog term van de vorm:

¹Het is dus perfect mogelijk in mijn parser dat er helemaal geen spaties gebruikt worden. Er is namelijk geen dubbelzinnigheid mogelijk. Wel moeten er newlines zijn, anders weten we niet wanneer dat commentaar stopt.

```
module(Start, Data, Funcs)
```

die rechtstreeks gebruikt kan worden door de interpreter en de analyse engine, zonder dat er nog verdere syntactische verwerking nodig is. Het voorbeeld voor het hogerlager.pwat bestand kan men in de bijlage vinden.

Om de lezer snel inzicht te geven in de implementatie, volgen hier twee korte en representatieve fragmenten uit `src/parser.pl` (DCG-regels):

```
1 % Module
2 module(module(Start, Data, Funcs)) -->
3     ws, "(", "module", ws,
4     start(Start),
5     data_segments(Data),
6     functions(Funcs),
7     ws, ")", ws.
8
9 % Numeric instruction (voorbeeld)
10 instruction(i32_const(X)) --> "i32.const", ws, integer(X), ws.
11 instruction(i32_add) --> "i32.add", ws.
```

Foutafhandeling: de parser-predicaat `parse/2` gebruikt `once(phrase(module(Module), Codes))`. Wanneer de input niet door de DCG geaccepteerd wordt, faalt `parse/2` en wordt dit doorgegeven naar de caller. In de CLI-flow leidt een `parse-failure` uiteindelijk tot een foutmelding en exit (zie `src/main.pl` en `src/runner.pl`). Testen die parser-fouten verwachten zijn onderdeel van `tests/test_parsing.pl`.

4 Werking van de Interpreter (run modus)

De uitvoering van een programma start in `run_file/2`. Na het parsen van het pwat-bestand wordt de bekomen module doorgegeven aan `run_module/2`. Daar worden eerst de data-segmenten omgezet naar een geheugenvoorstelling via `build_memory/2`. Vervolgens wordt de startfunctie van de module uitgevoerd.

De kern van de interpreter zit in het volgende predicaat:

```
execute_function(Index, Args, Module, Memory, Returns, Status, ContextIn, ContextOut) :-
    setup_function(Index, Args, Module, LocalsIn, Instrs, ResultCount),
    run_function(Instrs, Module, LocalsIn, Memory, Stack1, Signal, ContextIn, ContextOut),
    handle_function_signal(Signal, ResultCount, Stack1, Returns, Status).
```

De concrete implementatie en verdere predicaten staan in `src/function_runner.pl`. De CLI-entry en dispatch-logica staat in `src/main.pl` en de functies `run_file/2` en `analyse_file/4` in `src/runner.pl` zorgen voor het opbouwen van geheugen en het starten van `execute_function/8`. Traps versus normale returns: wanneer `exec_instruction/10` een foutsignaal (`trap`) teruggeeft, dan wordt dit doorgegeven tot `handle_function_signal/5` en resulteert het eindstatus-argument in `trap` (zie `src/function_runner.pl`: `handle_function_signal(trap, ..., [], trap)` en `prepare_to_return/6`). Voor ongeldige programma's gebruiken we de status `invalid`. Bij een normale finish is het eindstatus `finished` en worden de bovenste returnwaarden van de stack gehaald. Fouten in de uitvoering die leiden tot een ongeparste of niet-uitvoerbare module worden door `main.pl` afgevangen en resulteren in een foutmelding op `stderr` en `exit(1)`.

De analyse-context en instructielimiet worden hieronder verder toegelicht.

Opmerking over de stack: de startfunctie begint met een lege operand-stack, en elke functie-uitvoering krijgt een eigen lege stack. Dit volgt de WebAssembly-semantiek waarbij argumenten van de caller-stack worden gepopt bij een `call` en de returnwaarden nadien terug op de caller-stack geplaatst worden. Daardoor is er conceptueel geen globale operand-stack die doorheen

alle functies doorloopt, maar de zichtbare effecten bij een call zijn identiek aan de specificatie. Ik vermeld dit omdat ik niet zeker wist ofdat dit ok was volgens de opgave. Want de opgave spreekt precies over 1 globale stack, wat ik dus verwarrend vond.

Dit predicaat vormt de volledige levenscyclus van een functie-uitvoering:

- `setup_function/6` haalt de juiste functie uit de module, controleert het aantal argumenten en initialiseert de locals.
- `run_function/9` start de effectieve uitvoering van de instructies met een lege stack.
- `handle_function_signal/5` verwerkt het eindsignaal en haalt de gevraagde returnwaarden van de stack.

Sequentiële uitvoering van instructies

De uitvoering van instructies gebeurt in `exec_instructions/10`. Deze predicate doorloopt de instructielijst 1 voor 1. Voor elke instructie wordt `exec_instruction/10` aangeroepen, dat: de stack aanpast, locals leest of wijzigt, en een *signaal* teruggeeft dat bepaalt wat er verder moet gebeuren. Na elke instructie wordt via `continue_or_stop/11` beslist of de volgende instructie uitgevoerd moet worden of dat de uitvoering moet stoppen, dit gebeurt adhv signalen die worden doorgegeven.

Signaal-mechanisme

Elke instructie kan 1 van de volgende signalen teruggeven:

- `continue` : ga verder met de volgende instructie
- `returned` : stop de functie-uitvoering
- `trap` : er is een fout opgetreden
- `invalid` : ongeldig programma (bv. stack-underflow of ongeldige local)
- `break(N)` : verlaat een block of loop op niveau *N*

Deze signalen worden naar boven doorgegeven zodat blocks, loops en functies correct kunnen reageren.

Stack en locals

Alle berekeningen gebeuren via een stack, volledig analoog aan WebAssembly:

- constanten worden gepusht met `i32_const`,
- binaire operaties poppen twee waarden en pushen het resultaat,
- `local_get`, `local_set` en `local_tee` manipuleren de locals via de stack.

Indien een instructie onvoldoende waarden op de stack vindt of een ongeldige local aanspreekt, wordt een `invalid` gegenereerd.

Control flow: block, loop, if en branches

Control-flow instructies worden recursief afgehandeld:

- block verwerkt `break(N)` signalen via `handle_block_signal/6`.
- loop wordt uitgevoerd via `run_loop/11` en herstart zichzelf bij `break(0)`.
- if kiest het juiste instructieblok op basis van de stackwaarde.
- `br` en `br_if` genereren `break(N)` signalen.

Een belangrijk aspect in deze implementatie is dat control-flow niet met expliciete continuations wordt gemodelleerd, maar via het normale backtrackingmechanisme. Zodra een `break(N)` optreedt, wordt verdere matching binnen het huidige control-flow niveau effectief afgebroken. Predicaten die op een break-sigitaal matchen voeren doorgaans geen verdere recursie of alternatieve keuzes meer uit, waardoor ook de bijhorende keuze-punten worden afgesloten. Hierdoor stopt niet alleen de logische voortzetting van de evaluatie, maar wordt ook backtracking binnen dat control-flow niveau verhinderd.

Functie-aanroepen

Bij een `call` instructie wordt onderscheid gemaakt tussen:

- interne functies (via index) die recursief `execute_function/8` oproepen,
- primitieve functies die worden afgehandeld door `primitive_runner.pl`.

Argumenten worden van de stack gepopt en returnwaarden worden terug op de stack geplaatst.

Primitieve functies en geheugen

Primitieve functies zoals `print`, `println`, `print_int`, `read_int` en `rand_int` gebruiken het geheugen dat werd opgebouwd uit de data-segmenten.

Context

De parameters `ContextIn` en `ContextOut` worden doorheen de volledige uitvoering meegegeven. In *run modus* is deze context gewoon het atoom `run` en heeft deze geen invloed op de uitvoering. In *analyse modus* bevat deze context bijkomende informatie. Dat zullen we later bespreken. Hierdoor kan 'exact' dezelfde interpreter zowel voor effectieve uitvoering als voor analyse en padverkenning gebruikt worden.

5 Analyse en Paths

De analyse- en paths-modi hergebruiken exact dezelfde uitvoeringsemantiek als de `run`-modus, maar met een speciale context-term die extra informatie bijhoudt. In de CLI wordt deze context opgebouwd in `src/main.pl`:

```
1 % voorbeeld uit main.pl
2 ContextIn = analyse(MaxInstructions, 0, [], [], []).
```

De context-term heeft de vorm `analyse(MaxInstructions, CurrentCounter, Inputs, BadInputs, PathTrace)`. Tijdens uitvoering worden de velden geüpdatet door `src/function_runner.pl`:

- `increment_instruction_counter/2` verhoogt `CurrentCounter` na elke instructie.

- `record_branch_decision/3` voegt beslissingen toe aan `PathTrace` zodat de gemaakte branch-keuzes gelogd worden.

De limiet `MaxInstructions` voorkomt dat oneindige of extreem lange paden onbeperkt worden onderzocht: zodra `CurrentCounter >= MaxInstructions` is, stopt de huidige uitvoering.

In `src/main.pl` worden de resultaten van analyse en paths verder gefilterd, bijvoorbeeld met predicaten als `prefer_complete_trap_paths/2`, `suppress_shadowed_traps/2` en `keep_first_input_per_path/2` die zorgen dat we per uitvoeringspad slechts één representatieve input teruggeven.

Einde van de functie

Wanneer de instructies afgelopen zijn of een `return` optreedt, worden via `handle_function_signal/5` de gevraagde returnwaarden van de stack gehaald. Indien ergens een `trap` optreedt, wordt de volledige uitvoering onmiddellijk stopgezet.

6 Teststrategie

De teststrategie is opgebouwd rond kleine, gerichte `pwat`-programma's. In plaats van slechts een paar grote scenario's te testen, hebben we veel verschillende inputprogramma's geschreven die telkens maar 1 onderdeel van het systeem checken/gebruiken. Daardoor is een fout veel sneller te vinden: als een test faalt, is meteen duidelijk of het probleem in de parser, de interpreter, de analyse of de padverkenning zit.

De tests zijn opgedeeld in verschillende delen:

- **Parser-tests:** controleren of een bestand correct naar de interne prolog-structuur wordt omgezet. Hier testen we onder andere blokken, data-segmenten, lussen en geneste control-flow.
- **Run-tests:** voeren concrete programma's uit en controleren de eindtoestand en returnwaarden. Deze tests testen onder andere rekenkundige instructies, locals, branches, returns en geheugenoperaties.
- **Analyse-tests:** controleren of de analysemodus inputs kan vinden die tot een trap leiden. Hier testen we onder meer `unreachable`, deling door nul, conditionele traps en lussen met een instructielimiet.
- **Paths-tests:** controleren of voor elk uitvoeringspad minstens 1 geschikte input gevonden wordt, ook wanneer een programma meerdere geneste vertakkingen bevat.
- **CLI-tests:** testen het volledige programma via de command-line interface en controleren of de output exact in het verwachte formaat verschijnt.

Alle tests worden uitgevoerd via een testscript. Voor de meeste gevallen gebruiken we kleine tijdelijke programma's in de test zelf, zodat we per test exact kunnen bepalen welk gedrag verwacht wordt. Voor integratietests gebruiken we ook bestaande voorbeeldbestanden uit de map `examples/`, zodat dezelfde programma's zowel handmatig als automatisch nagekeken kunnen worden.

A Bijlage

Voorbeeld internevoorstelling na parsing voor het hogerlage.pwat bestand.

```
1 module(0,
2   [data(0,"
      \71\101\101\102\32\101\101\110\32\103\101\116\97\108\32\105\110\58")
      ,
3     data(18,"\104\111\103\101\114"),
4     data(23,"\108\97\103\101\114"),
5     data(28,"\99\111\114\114\101\99\116")]],
6   [func(0,2,0,
7     [i32_const(0),
8       i32_const(18),
9       call(println),
10      i32_const(0),
11      i32_const(100),
12      call(rand_int),
13      local_set(0),
14      loop([
15        i32_const(0),
16        i32_const(100),
17        call(read_int),
18        local_set(1),
19        local_get(1),
20        local_get(0),
21        i32_lt_s,
22        if(
23          [i32_const(18), i32_const(5), call(println), br(1)],
24          [local_get(1), local_get(0), i32_gt_s,
25            if(
26              [i32_const(23), i32_const(5), call(println), br(2)],
27              [i32_const(28), i32_const(7), call(println), br(2)]
28            )
29          ]
30        )
31      ]),
32    ]),
33  ])
```